

Mini Paper — 2.1 Elements of Computational Thinking

Time: ~30 minutes · Total: 25 marks · Answer all questions.

Mark your work against the mark scheme at the end; aim for ~1 minute per mark.

Questions

Scenario. *FastPark* is a company building software for a multi-storey car park. Drivers enter at a barrier, take a digital ticket, park, and pay at a machine before leaving. A companion mobile app lets drivers search nearby car parks, see live space availability, and reserve a bay in advance. The software must control the entry/exit barriers, track how many spaces are free, calculate charges, and serve the app.

Q1 [3] (AO2). The developers model the car park abstractly in software. (a) State what is meant by *abstraction*. [1] (b) Give **one** real-world detail about the car park that would be *removed* in the model, and **one** detail that must be *kept*, explaining why each choice is appropriate. [2]

Q2 [4] (AO2). The companion app must show "spaces free" for each car park. (a) Identify **two inputs** the availability feature needs and **two outputs** it produces. [2] (b) State **one precondition** that must be true before a driver can reserve a bay, and explain why it is needed. [2]

Q3 [4] (AO3). Live space-availability data for each car park is requested by many drivers' apps every few seconds. Explain how **caching** this data on the server could improve performance, and **evaluate one trade-off** of doing so in this scenario. [4]

Q4 [4] (AO2). When a driver pays at the exit machine, the software runs through several steps in a fixed order. (a) List **four** sub-procedures the payment-and-exit process could be decomposed into, in the correct **order**. [3] (b) Explain why the order of two of your steps cannot be swapped. [1]

Q5 [4] (AO2). Describe **two** decision points the exit software must handle, and for **each** give the exact **condition** (a Boolean test) and the outcome of **both** branches. [4]

Q6 [6] (AO3). When a driver opens the app's home screen, the software must (a) fetch the list of nearby car parks, (b) fetch live availability for each, (c) load the user's saved payment cards, and (d) render the map background. Discuss which of these tasks could be performed **concurrently**, and evaluate the **benefits and trade-offs** of using concurrent processing here. [6]

Mark scheme

Total: 25 marks

Q1 [3] - (a) Abstraction = removing/hiding unnecessary detail so only information relevant to the problem remains. [1] - (b) One detail removed with reason (1): e.g. the colour/material of the walls, lighting, CCTV layout — irrelevant to tracking spaces or charging. [1] - One detail kept with reason (1): e.g. the number of bays / which bays are occupied / the tariff — essential to calculating availability and charges. [1] - *Examiner tip:* the marks are for the *justification* ("because it is/ isn't relevant to this problem"), not just naming a detail.

Q2 [4] - (a) Two inputs (1 each, max 2): barrier entry/exit sensor counts, a bay sensor signal, the driver's chosen car park / location, reservation request. **[2]** - (b) Precondition (1): e.g. at least one free bay must exist / the driver must be logged in / the chosen car park must be open — must be a condition true *before* reserving. Reason (1): so the reservation cannot be made against a full/closed car park (avoids invalid bookings). **[2]** - *Examiner tip*: an input is data *in* (a sensor reading), not a process like "count the cars" — don't confuse the two.

Q3 [4] - Caching explained (up to 2): store the computed "spaces free" figure in fast memory on the server (1); serve repeated app requests from the cache instead of recounting/querying sensors each time → faster response and less load on the database/sensors (1). **[2]** - Trade-off evaluated (up to 2): the cached count can become **stale** because cars enter/leave continuously (1), so a driver may see a space that is already taken; it must be refreshed/invalidated frequently (e.g. every few seconds), and caching uses extra memory — a judgement that a short refresh interval balances speed against accuracy (1). **[2]** - *Examiner tip*: "evaluate" needs both the benefit *and* a weighed downside (stale data) with a conclusion — a one-sided answer caps at 2.

Q4 [4] - (a) Four ordered sub-procedures (3): award up to 3 for a sensible decomposition in correct order, e.g. `readTicket()` → `calculateCharge()` → `takePayment()` → `openExitBarrier()` (also accept `printReceipt()`, `updateSpaceCount()`). Full 3 needs four named, correctly-ordered steps; 2 for minor order/naming slips. **[3]** - (b) Order justification (1): a data dependency — e.g. you cannot `takePayment()` before `calculateCharge()` because the amount isn't known, or cannot `openExitBarrier()` before payment is confirmed. **[1]** - *Examiner tip*: name *scenario-relevant* sub-procedures and show one step depends on the output of an earlier one.

Q5 [4] - Two decision points, each with a specific Boolean condition + both outcomes (2 marks each, max 4). Examples: - *Is payment sufficient?* IF `amountPaid >= chargeDue` → open barrier / give change; ELSE → request more money / reject. **[2]** - *Is the ticket valid?* IF `ticketExists AND NOT alreadyPaid` → process exit; ELSE → show error / call attendant. **[2]** - (Also accept: *grace period* IF `timeInCarPark <= freeMinutes` → charge £0 else charge tariff.) - *Examiner tip*: state the actual Boolean test (use `>=`, `AND`) and what happens on **both** the true and false branch — vague "if enough money" loses marks.

Q6 [6] Indicative content — credit a reasoned, applied discussion: - Tasks (a), (c) and (d) are independent (no task needs another's output) → candidates for concurrent execution; (b) "fetch availability for each car park" depends on (a) having returned the list, so it must follow (a) but the individual fetches can then run concurrently with each other (1–2). - Benefit: the home screen loads/appears far faster and stays responsive because fetches and rendering happen in parallel rather than one-after-another; better use of multi-core hardware and the network (1–2). - Trade-offs: harder to design/program/debug; results must be combined when all arrive; contention for limited mobile network bandwidth means speed-up is not linear; on a single core tasks are only time-sliced, not truly parallel; risk of a race condition if shared data is updated (1–2). - Justified judgement: concurrency is worthwhile here because most tasks are independent and I/O-bound, but the developer must respect the (a) → (b) dependency and manage synchronisation (1). - Bands: 5–6 = sustained, balanced discussion applied to the app with a justified conclusion (incl. the dependency); 3–4 = some application + some trade-offs; 1–2 = generic points with little scenario link. - *Examiner tip*: always check for a data dependency before claiming tasks are concurrent — spotting that (b) must wait for (a) is what separates the top band.